Conversione tra le basi binarie

Unità di Apprendimento 1 · Lezione 5

La rappresentazione delle informazioni

Tecnologie e Progettazione di Sistemi Informatici e di Telecomunicazioni (TPSI)

Classe 3ª – Istituto Tecnico Informatico

In questa lezione impareremo

1. La conversione tra **binario e ottale**
2. La conversione tra **binario ed esadecimale**
3. La conversione tra **ottale ed esadecimale**

> Il trucco: niente divisioni né moltiplicazioni! Basta **raggruppare i bit** e usare una tabellina.

Introduzione: un caso particolare

Affronteremo casi particolari che permettono di sfruttare alcune **proprietà matematiche** per passare tra basi con caratteristiche comuni.

Consideriamo:

Sistema	Base	Come potenza di 2
binario	b = 2	$2^1 = 2$
ottale	b = 8	$2^3 = 8$
esadecimale	b = 16	$2^4 = 16$

Il legame tra le basi

Tra queste basi esiste un **particolare legame**: sono legate da una **potenza**.

- la base **ottale** è la **terza potenza** della base binaria
- la base **esadecimale** è la **quarta potenza** della base binaria

Relazione generale: $B_a = B_b^{k}$

> Quando tra due basi vale $B_a = B_b^{k}$, ogni cifra della base più grande (B_a) si scrive con esattamente **k cifre** della base più piccola (B_b).

Quante cifre binarie per ogni cifra?

Ottale ↔ binario: $B_1 = 8$, $B_2 = 2$ → **k = 3** perché $8 = 2^3$

→ ogni cifra ottale = **3 bit**

Esadecimale ↔ binario: $B_1 = 16$, $B_2 = 2$ → **k = 4** perché $16 = 2^4$

→ ogni cifra esadecimale = **4 bit** (un *nibble*)

Base	Cifre	Bit per cifra	Combinazioni
Ottale	0 … 7	3	$2^3 = 8$
Esadecimale	0 … F	4	$2^4 = 16$

1. Binario ↔ ottale

Premessa: la tabella ottale

Il sistema ottale è usato principalmente come rappresentazione **intermedia** per la comunicazione con le macchine digitali.

ottale	0_8	1_8	2_8	3_8	4_8	5_8	6_8	7_8
decimale	0	1	2	3	4	5	6	7
binario	**000**	**001**	**010**	**011**	**100**	**101**	**110**	**111**

Due osservazioni

1. La codifica ottale **coincide** con i primi otto elementi della codifica decimale
2. Tutte le otto cifre sono codificate in binario con **stringhe di 3 bit**, perché $2^3 = 8$

> **Regola: da binario a ottale**
> Raggruppa le cifre binarie **a gruppi di 3** (partendo da destra) e sostituisci ogni gruppo con la corrispondente cifra ottale.

Consiglio: i pesi dentro ogni gruppo di 3 bit sono **4 · 2 · 1**. Ad esempio `110` = 4 + 2 + 0 = **6**.

Binario → ottale: 110111_2

Separiamo i bit a gruppi di 3:

$110_2 \quad 111_2$

Convertiamo in ottale ogni gruppo:

$110_2 = 6_8$ e $111_2 = 7_8$

Quindi:

> $110111_2 = 67_8$

Schema sintetico:

```
  110    111
   ↓      ↓
   6      7
```

Binario → ottale: 101100_2

```
101    100
 ↓      ↓
 5      4
```

> $101100_2 = 54_8$

Verifica in decimale: $101100_2 = 32 + 8 + 4 = 44 \cdot 54_8 = 5 \cdot 8 + 4 = 44$ ✓

Se il numero non è multiplo di 3

Se il numero binario **non ha un numero di bit multiplo di 3**, bisogna aggiungere **a sinistra** i bit mancanti, naturalmente con valore **0** (non cambiano il valore).

Esempio: 11001010_2 (8 bit → ne mancano 1)

```
011     001     010          ← aggiunto uno 0 a sinistra
 ↓       ↓       ↓
 3       1       2
```

> $11001010_2 = 312_8$

Procedura in tre passi

Esempio: 1011010_2 (7 bit)

1. **Separiamo** i bit a gruppi di tre **partendo da destra**:

   `1  011  010`

2. **Completiamo** le terne aggiungendo a sinistra gli 0 mancanti:

   `001  011  010`

3. **Convertiamo** ogni terna:

```
001    011    010
 ↓      ↓      ↓
 1      3      2
```

$1011010_2 = 132_8$

Si parte sempre **da destra**: gli zeri di completamento vanno sempre a **sinistra**, mai a

Numeri frazionari: binario → ottale

Anche per i numeri reali i bit devono essere raggruppabili in **terzetti**, aggiungendo eventualmente:

- uno o più **zeri a sinistra** (cifra più significativa) per la **parte intera**
- uno o più **zeri a destra** (cifra meno significativa) per la **parte frazionaria**

Esempio: 1011010.11_2

- parte intera: `001 011 010` (si aggiungono zeri **a sinistra**)
- parte frazionaria: `110` (si aggiunge uno zero **a destra**)

```
001    011    010   .   110
 ↓      ↓      ↓         ↓
 1      3      2   .     6
```

$1011010.11_2 = 132.6_8$

Un altro esempio frazionario

Convertiamo 11111010.1_2 in base 8.

Separiamo la parte intera da quella frazionaria:

- parte intera: `11111010`
- parte frazionaria: `1`

Aggiungiamo gli zeri mancanti:

- parte intera: `011 111 010`
- parte frazionaria: `100`

```
  011    111    010   .   100
   ↓      ↓      ↓         ↓
   3      7      2     .   4
```

$11111010.1_2 = 372.4_8$

Da ottale a binario

Per passare dalla codifica ottale a quella binaria, a **ogni cifra ottale** sostituiamo la corrispondente **cifra codificata in binario** (3 bit).

Esempio: 63_8

```
  6      3
  ↓      ↓
110    011
```

> $63_8 = 110011_2$

Da ottale a binario: 126_8

```
  1      2      6
  ↓      ↓      ↓
 001    010    110
```

Otteniamo: $126_8 = 001010110_2$

Gli **zeri a sinistra** possono essere eliminati (sono "superflui"):

$126_8 = 1010110_2$

Attenzione: si tolgono **solo gli zeri a sinistra** del numero intero, mai quelli in mezzo!

Esercizi: binario ↔ ottale

Converti:

a. 101111_2 → ottale	b. 110101101_2 → ottale	c. 1010101_2 → ottale
d. 57_8 → binario	e. 435_8 → binario	f. 10.1_2 → ottale

Soluzioni: a. 57_8 · b. 655_8 · c. 125_8 · d. 101111_2 · e. 100011101_2 · f. 2.4_8

2. Binario ↔ esadecimale

Premessa: la tabella esadecimale

Il sistema esadecimale è usato come rappresentazione **intermedia** per la comunicazione con le macchine digitali.

Esa	0	1	2	3	4	5	6	7
Dec	0	1	2	3	4	5	6	7
Bin	0000	0001	0010	0011	0100	0101	0110	0111
Esa	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
Dec	8	9	10	11	12	13	14	15
Bin	1000	1001	1010	1011	1100	1101	1110	1111

Perché l'esadecimale è così comodo

- Ogni **byte** è composto da 8 bit → si suddivide in **due gruppi da 4 bit (nibble)**
- Ciascun nibble è codificato con **una cifra esadecimale**
- Quindi **un byte = due cifre esadecimali**

> **Regola: da binario a esadecimale**
> Raggruppa le cifre binarie **a gruppi di 4** (partendo da destra) e sostituisci ogni gruppo con la corrispondente cifra esadecimale.

Esempi d'uso: colori web `#FF8800` (3 byte), indirizzi MAC `3C:52:82:…`, indirizzi di memoria `0x7FFE`.

Binario → esadecimale: 10101110_2

Separiamo i bit a gruppi di **4**:

$1010_2 \quad 1110_2$

Convertiamo ogni gruppo:

$1010_2 = A_{16}$ e $1110_2 = E_{16}$

```
  1010   1110
   ↓      ↓
   A      E
```

> $10101110_2 = AE_{16}$

Verifica: $10101110_2 = 128+32+8+4+2 = 174$ · $AE_{16} = 10 \cdot 16 + 14 = 174$ ✓

Se non è multiplo di 4: 111001_2

I bit sono 6: ne mancano 2 per arrivare a 8. Si aggiungono **zeri a sinistra**:

```
  0011   1001
   ↓      ↓
   3      9
```

> $111001_2 = 39_{16}$

Verifica: $111001_2 = 32+16+8+1 = 57 \cdot 39_{16} = 3 \cdot 16 + 9 = 57$ ✓

Numeri frazionari: binario → esadecimale

Convertiamo il numero binario reale 111010.1_2.

Completiamo con gli zeri mancanti per formare i singoli **nibble**:

- parte intera: zeri **a sinistra** → `0011 1010`
- parte frazionaria: zeri **a destra** → `1000`

```
  0011   1010   .   1000
    ↓      ↓          ↓
    3      A      .   8
```

$111010.1_2 = 3A.8_{16}$

Da esadecimale a binario

Per passare dalla codifica esadecimale a quella binaria, a **ogni cifra esadecimale** sostituiamo la corrispondente cifra codificata in binario (**4 bit**).

Esempio: 63_{16}

```
   6      3
   ↓      ↓
0110   0011
```

> $63_{16} = 01100011_2$

Esempio: $C8_{16}$

```
   C      8
   ↓      ↓
1100   1000
```

Esadecimale frazionario → binario

Convertiamo $1F.4_{16}$ in binario.

```
   1      F     .    4
   ↓      ↓          ↓
 0001   1111    .  0100
```

Il risultato è: $1F.4_{16} = 0001\ 1111.0100_2$

Dopo aver eliminato gli zeri "superflui" (a sinistra della parte intera e a destra della frazionaria):

> $1F.4_{16} = 11111.01_2$

Esercizi: binario ↔ esadecimale

Converti:

a. 11110000_2 → esa	b. 10110111_2 → esa	c. 1101.11_2 → esa
d. $A5_{16}$ → binario	e. $B7_{16}$ → binario	f. $2.C_{16}$ → binario

Soluzioni: a. $F0_{16}$ · b. $B7_{16}$ · c. $D.C_{16}$ · d. 10100101_2 · e. 10110111_2 · f. 10.11_2

3. Ottale ↔ esadecimale

Passare attraverso il binario

Il metodo più veloce per passare dal sistema **ottale** all'**esadecimale** (o viceversa) è **passare attraverso il sistema binario**.

```
Ottale  ──▶  Binario  ──▶  Esadecimale
(3 bit)    (raggruppo     (4 bit)
           diversamente)
```

Procedura

1. **Ottale → binario:** ogni cifra ottale diventa 3 bit
2. **Riraggruppa** i bit a gruppi di **4** (da destra)
3. **Binario → esadecimale:** ogni gruppo diventa una cifra esa

(Viceversa: ogni cifra esa → 4 bit → riraggruppa a gruppi di 3 → cifre ottali.)

Ottale → esadecimale: 352_8

```
  3      5      2          (ottale: 3 bit ciascuno)
  ↓      ↓      ↓
 011    101    010         → 011101010
                             riraggruppo a 4 da destra:
    1110    1010
      ↓       ↓
      E       A
```

$(352)_8 = (1110\ 1010)_2 = (EA)_{16}$

Verifica: $352_8 = 3 \cdot 64 + 5 \cdot 8 + 2 = 234 \cdot EA_{16} = 14 \cdot 16 + 10 = 234$ ✓

Ottale → esadecimale: 743_8

```
    7        4        3          (ottale: 3 bit ciascuno)
    ↓        ↓        ↓
   111      100      011         → 111100011  (9 bit)

riaggruppo a 4 da destra (servono 3 zeri a sinistra):
0001    1110    0011
  ↓       ↓       ↓
  1       E       3
```

$(743)_8 = (0001\ 1110\ 0011)_2 = (1E3)_{16}$

Verifica: $743_8 = 448 + 32 + 3 = 483 \cdot 1E3_{16} = 256 + 14{\cdot}16 + 3 = 483$ ✓

Esadecimale → ottale: $7C_{16}$

```
  7        C
  ↓        ↓
0111     1100        → 01111100
                        riraggruppo a 3 da destra (uno 0 a sinistra):
001    111    100
 ↓      ↓      ↓
 1      7      4
```

$(7C)_{16} = (001\ 111\ 100)_2 = (174)_8$

Verifica: $7C_{16} = 7 \cdot 16 + 12 = 124 \cdot 174_8 = 64 + 56 + 4 = 124$ ✓

Esadecimale → ottale: $2AC_{16}$

```
   2          A          C
   ↓          ↓          ↓
 0010       1010       1100      → 001010101100
                                     riraggruppo a 3 da destra:
 001    010    101    100
  ↓      ↓      ↓      ↓
  1      2      5      4
```

$(2AC)_{16} = (001\ 010\ 101\ 100)_2 = (1254)_8$

Verifica: $2AC_{16} = 512 + 160 + 12 = 684$ · $1254_8 = 512 + 128 + 40 + 4 = 684$ ✓

Riepilogo dei raggruppamenti

Da → A	Passaggi	Gruppi
Binario → Ottale	raggruppa i bit	3
Ottale → Binario	espandi ogni cifra	3
Binario → Esadecimale	raggruppa i bit	4
Esadecimale → Binario	espandi ogni cifra	4
Ottale ↔ Esadecimale	passa dal binario	3 poi 4 (o viceversa)

Regole d'oro

- si raggruppa **da destra** verso sinistra (parte intera)
- nella **parte frazionaria** si raggruppa **dalla virgola verso destra**
- gli zeri di completamento vanno **a sinistra** (intera) e **a destra** (frazionaria)

Esercizi: ottale ↔ esadecimale

Converti passando dal binario:

a. 435_8 → esa	b. 777_8 → esa	c. $F3_{16}$ → ottale
d. $1A9_{16}$ → ottale	e. 63_8 → esa	f. FF_{16} → ottale

Soluzioni: a. $11D_{16}$ · b. $1FF_{16}$ · c. 363_8 · d. 651_8 · e. 33_{16} · f. 377_8

Esempio svolto (c): $F3_{16} = 1111\ 0011_2 \rightarrow 011\ 110\ 011 \rightarrow 363_8$

Errori comuni da evitare

1. **Raggruppare da sinistra** invece che da destra (parte intera)
2. **Aggiungere zeri dalla parte sbagliata** (a destra della parte intera!)
3. **Usare gruppi di 4 per l'ottale** (o di 3 per l'esadecimale)
4. **Dimenticare** di riraggruppare quando si passa da ottale a esadecimale
5. **Scambiare le lettere:** A = 10 = `1010`, B = 11 = `1011`, … F = 15 = `1111`

Trucco per la tabella esa: basta ricordare i pesi **8 · 4 · 2 · 1** dentro ogni nibble. Es. `1101` = 8+4+0+1 = 13 = **D**.

Riepilogo

- **Ottale** = base 2^3 → **1 cifra ottale = 3 bit**
- **Esadecimale** = base 2^4 → **1 cifra esa = 4 bit**
- **Binario → ottale/esa:** raggruppa i bit (3 o 4) e sostituisci
- **Ottale/esa → binario:** espandi ogni cifra in 3 o 4 bit
- **Ottale ↔ esa:** passa sempre dal **binario**
- **Zeri di completamento:** a sinistra (parte intera), a destra (parte frazionaria)

Prossimo argomento: operazioni aritmetiche nei sistemi di numerazione binari.

Grazie per l'attenzione

Domande?